UNIVERSIDAD MAYOR
Facultad de Ciencias, Ingeniería y Tecnología · Escuela de Ingeniería Civil

INGT1003 Arquitectura de Desarrollo (Móvil y Web)

ARTICLA

Propuesta de valor y arquitectura

Asistente legal inteligente construido sobre la legislación chilena y los libros de la Facultad de Derecho de la Universidad de Chile

Hito 1 · La propuesta que se vende

Documento de propuesta comercial y técnica

Equipo: Articla
Integrantes (orden alfabético por apellido): Matías del Fierro · Camilo González
Docente: Prof. Carlos Muñoz S.

Santiago de Chile, septiembre de 2026

Índice

1. Introducción..... 1
2. El problema y su evidencia..... 1
3. La promesa..... 2
4. Arquitectura de información..... 2
5. Arquitectura cliente-servidor y capas..... 3
6. Selección de lenguajes y frameworks..... 4
7. Prototipo navegable..... 7
8. Conclusión..... 10
9. Bibliografía..... 11
Anexo A. Equipo y roles..... 12
Anexo B. Dominios de datos y fuentes previstas..... 12
Anexo C. Declaración de uso de inteligencia artificial..... 12

1. Introducción

En Chile, entender un derecho o saber si una situación cotidiana tiene respaldo legal suele depender de poder pagar una hora de abogado o de conseguir un cupo en una Corporación de Asistencia Judicial. Mientras la persona no obtiene esa orientación, un plazo puede vencer, un aumento de arriendo puede aplicarse sin que el inquilino sepa si es legal, o un finiquito puede firmarse sin que el trabajador sepa qué le corresponde realmente.

Articla nace para acortar esa distancia. Es un asistente legal conversacional que responde preguntas de orientación jurídica en lenguaje simple, citando siempre el artículo exacto de la ley o el código que respalda la respuesta. La base de conocimiento se construye sobre los códigos y leyes vigentes de Chile y sobre los libros y apuntes de estudio de la Facultad de Derecho de la Universidad de Chile, de modo que cada respuesta pueda rastrearse hasta una fuente verificable.

Este documento presenta el problema que Articla resuelve, la evidencia que lo dimensiona, la experiencia que se promete al usuario, la arquitectura de información del producto y la arquitectura técnica cliente-servidor que sostiene esa promesa, junto con las decisiones de tecnología tomadas para el prototipo de este primer hito.

2. El problema y su evidencia

El acceso gratuito a orientación legal en Chile se concentra en pocos canales con una demanda muy alta. Solo en la Región del Biobío, la Corporación de Asistencia Judicial registró 192.049 atenciones de orientación e información durante 2019, una categoría que corresponde a consultas de orientación y no a representación en juicio (Corporación de Asistencia Judicial de la Región del Biobío, 2020). Esa cifra corresponde a una sola de las cuatro corporaciones que operan en el país, de modo que la demanda nacional es considerablemente mayor.

El problema no es solo de volumen de demanda, sino de comprensión: de acuerdo con el Informe de Caracterización de Usuarios y Usuarias del Poder Judicial (2019), solo un 21% de las personas sin formación legal es capaz de identificar correctamente la función del Poder Judicial, y un 26% declara que la información entregada por las instituciones es poco clara (Open Government Partnership, 2021). El mismo informe recoge una medición de la Dirección de Estudios Sociales UC de 2018 que resulta especialmente elocuente: un 57% de la ciudadanía cree que una persona de altos ingresos puede acceder a un juicio justo, frente a solo un 4% que cree lo mismo para una persona de bajos ingresos (Open Government Partnership, 2021).

La Encuesta Nacional de Necesidades Jurídicas y Acceso a la Justicia, desarrollada por GfK Adimark para el Ministerio de Justicia, confirma que buena parte de las necesidades jurídicas de la población civil, laboral y de consumo nunca llegan siquiera a plantearse ante una institución, precisamente porque la barrera de entrada —tiempo, dinero o simple desconocimiento de a quién acudir— es demasiado alta (Ministerio de Justicia y Derechos Humanos y GfK Adimark Chile, 2015).

A quién le duele: Camila, 34 años, trabajadora independiente en Santiago. Recibe una carta de su arrendador anunciando un alza de renta a mitad de contrato y no sabe si puede negarse. Pedir hora en la Corporación de Asistencia Judicial no le da una respuesta inmediata, y pagar una consulta privada no está entre sus prioridades esta quincena. Hoy, su alternativa real es preguntar en un grupo de Facebook del barrio o resignarse a pagar el alza sin saber si es exigible.

3. La promesa

Para cualquier persona en Chile que enfrenta una duda legal cotidiana y no tiene acceso rápido ni económico a orientación jurídica, **Articla entrega una respuesta clara en menos de un minuto, citando el artículo exacto de la ley chilena que la respalda.**

Esa frase abre la presentación oral y aparece palabra por palabra bajo el titular de la pantalla de inicio del prototipo y en el README del repositorio, de modo que la misma promesa (incluido el plazo de menos de un minuto) se presenta, se muestra y se mide en el Hito 3. La alternativa actual del usuario —pedir hora y esperar, preguntar en redes sociales, o pagar una consulta privada— es la vara con la que se mide el valor de Articla: no reemplaza a un abogado en un litigio, pero sí resuelve la primera pregunta, la que hoy no tiene una respuesta inmediata.

4. Arquitectura de información

El producto se organiza en cinco secciones desde la pantalla de inicio: Preguntar (el chat legal), Historial de consultas, Biblioteca de fuentes (transparencia sobre qué códigos, leyes y libros están previstos), Cuenta y Aviso legal. Cada una tiene su pantalla en el prototipo. Una consulta resuelta en el chat lleva a una respuesta con las citas visibles, y cada cita lleva al detalle de la fuente: el texto del artículo citado y su origen. La Biblioteca de fuentes lleva, además, a una ficha de cada fuente prevista (tipo, origen, licencia y estado de carga). La Figura 1 resume esa jerarquía y las rutas de navegación:

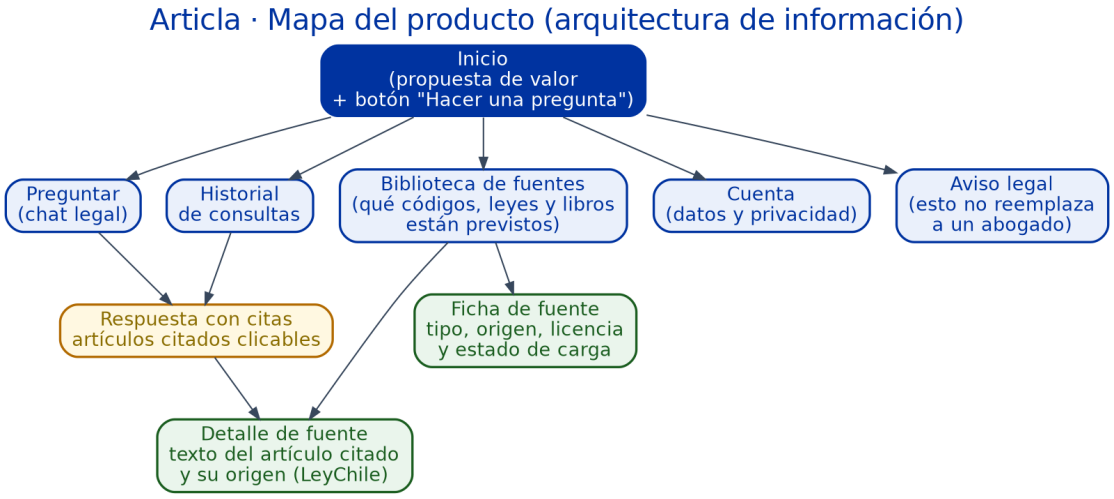


Figura 1. Mapa del producto — arquitectura de información de Articla.

El vocabulario de cada sección usa el lenguaje del usuario ("Preguntar", "Historial") y no el del sistema ("query", "logs"), siguiendo el mismo criterio que se aplicó al nombrar las pantallas del prototipo navegable.

5. Arquitectura cliente-servidor y capas

Articla se organiza como una arquitectura cliente-servidor con tres capas lógicas distribuidas entre ambos extremos: la capa de presentación vive en el cliente; la capa de negocio, que orquesta la recuperación de información y la generación de la respuesta, y la capa de datos, que aloja el corpus legal vectorizado y los datos relacionales del producto, viven en el servidor. El modelo de lenguaje no pertenece a ninguna de esas capas: es un servicio externo de un proveedor, al que el servidor llama por API (en la Figura 2, el recuadro punteado).

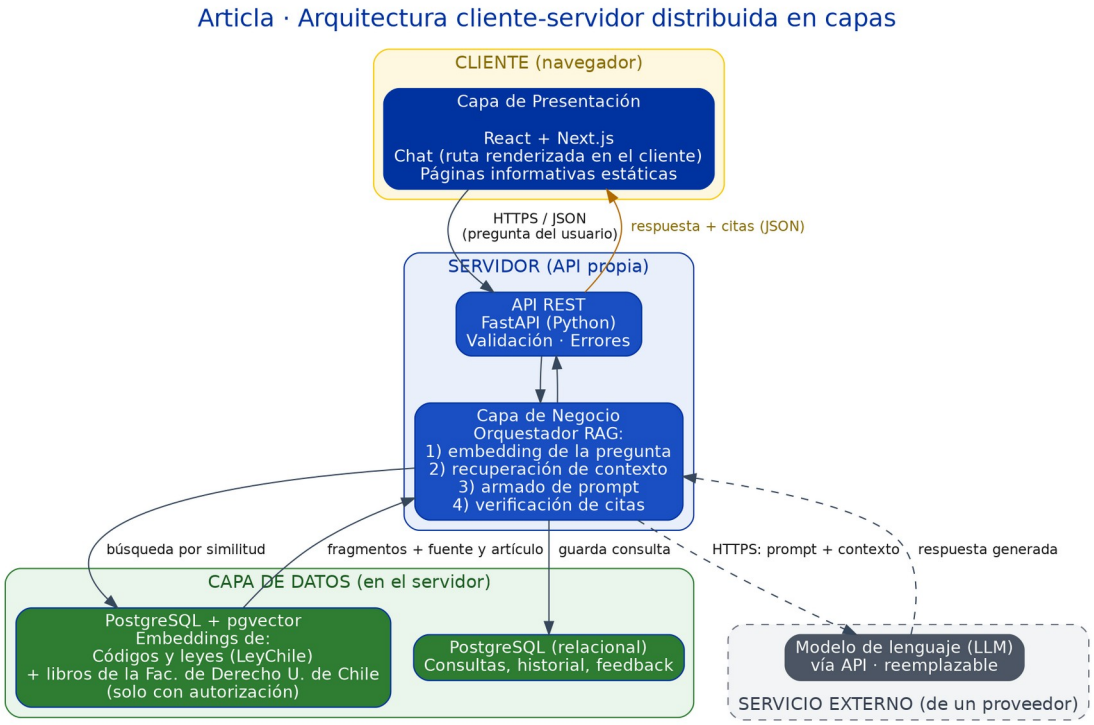


Figura 2. Arquitectura cliente-servidor distribuida en capas de Articla, con el modelo de lenguaje como servicio externo.

5.1 Recorrido de un caso de uso: "¿puede subirme el arriendo?"

Cuando la usuaria escribe su pregunta en el chat, la capa de presentación (React) la envía por HTTPS a la API REST del servidor. La capa de negocio calcula el embedding de la pregunta, lo compara por similitud contra los embeddings almacenados en PostgreSQL con la extensión pgvector y recupera los fragmentos más cercanos; en este caso, el artículo 1545 del Código Civil (Código Civil de Chile, s. f., art. 1545), según el cual lo pactado en un contrato obliga a las partes como una ley, y el artículo 1 de la Ley 18.101 (Ley 18.101, 1982, art. 1), que remite al Código Civil en lo que la ley especial de arrendamiento no regula. Con ese contexto arma un prompt en el que cada fragmento lleva un código, y lo envía al servicio de generación, que redacta la respuesta en lenguaje simple y debe indicar los códigos de los fragmentos en

que se apoya. Antes de devolverla, la capa de negocio verifica cada cita de forma mecánica: (1) el código citado debe estar entre los fragmentos recuperados para esa consulta, y si el modelo cita algo que no se le entregó, esa cita se descarta; (2) la respuesta debe quedar con al menos una cita válida, o se reemplaza por un aviso de que no se encontró respaldo en las fuentes; y (3) cada cita se muestra con el texto literal del artículo para que la usuaria pueda cotejarlo. Esta verificación asegura que lo citado existe y fue recuperado, pero no que el modelo lo haya interpretado bien; por eso la exactitud de fondo se mide con la batería de preguntas de prueba descrita en la conclusión. Finalmente, registra la consulta en la base relacional y devuelve la respuesta con sus citas, que el cliente muestra como chips clicables: al presionarlos, la usuaria navega al detalle de cada fuente.

Esta separación importa porque aísla responsabilidades que cambian a ritmos distintos: la interfaz puede rediseñarse sin tocar la lógica de recuperación, el motor de búsqueda puede cambiar de proveedor de embeddings sin tocar la interfaz, y el modelo de lenguaje puede reemplazarse sin rediseñar el resto del sistema.

6. Selección de lenguajes y frameworks

Se compararon al menos dos alternativas para cada decisión relevante, con tres criterios propios: curva de aprendizaje del equipo, ajuste al problema (RAG conversacional) y costo de operación. Se firma una decisión en cada caso, asumiendo su desventaja.

6.1 Frontend

Tabla 1. Comparación de alternativas de frontend.

Criterio	React + Next.js	Vue + Nuxt
Curva de aprendizaje del equipo	Media: abundante documentación y ejemplos disponibles para aprender.	Baja al inicio, pero con menos ejemplos específicos de interfaces de chat.
Ecosistema para UI conversacional	Amplio soporte de librerías de chat/streaming de texto.	Ecosistema más pequeño para interfaces de chat en tiempo real.
Renderizado híbrido (SPA + páginas informativas)	Next.js permite combinar SPA para el chat con páginas estáticas para landing y aviso legal.	Nuxt ofrece lo mismo, con menor cantidad de ejemplos documentados.
Costo de operación	Sin costo de licencia; un prototipo puede desplegarse en planes gratuitos.	Sin costo de licencia y con costo de despliegue equivalente: no diferencia.

Decisión (Tabla 1): React + Next.js. La desventaja que se asume es una configuración inicial más compleja que una SPA pura, a cambio de poder servir el inicio y el aviso legal como páginas estáticas indexables por buscadores, mientras el chat se renderiza en el cliente (ver 6.4).

6.2 Backend

Tabla 2. Comparación de alternativas de backend.

Criterio	Python + FastAPI	Node.js + Express
Ajuste al problema (RAG)	Ecosistema maduro de librerías de recuperación semántica y procesamiento de texto legal.	Ecosistema más orientado a APIs generales; menos librerías nativas de RAG.
Rendimiento en operaciones asíncronas	Soporte async nativo, adecuado para llamadas concurrentes al motor de embeddings y al LLM.	Buen soporte async, ampliamente probado en producción.
Documentación automática de la API	OpenAPI/Swagger generado automáticamente desde el código.	Requiere configurar manualmente una librería adicional.
Curva de aprendizaje del equipo	Media: exige usar Python junto a JavaScript, aunque su sintaxis y tipado son claros.	Baja: el mismo lenguaje (JavaScript) en cliente y servidor.
Costo de operación	Sin costo de licencia; recursos de servidor similares a los de Node.js para esta carga.	Sin costo de licencia; recursos de servidor similares.

Decisión (Tabla 2): Python + FastAPI. La desventaja asumida es que el equipo debe operar dos lenguajes distintos entre cliente y servidor (JavaScript y Python), lo que exige más disciplina de documentación en el contrato de la API, a cambio de acceder al mejor ecosistema disponible para construir el motor de recuperación semántica.

6.3 Almacenamiento vectorial

Tabla 3. Comparación de alternativas de almacenamiento vectorial.

Criterio	PostgreSQL + pgvector	Pinecone (servicio gestionado)
Costo de operación	Sin costo adicional: se usa la misma base de datos relacional del proyecto.	Plan gratuito limitado; costo creciente con el volumen del corpus legal.
Soberanía de los datos	El corpus legal permanece en infraestructura propia del equipo.	Los datos se almacenan en servidores de un proveedor externo.
Curva de aprendizaje	Requiere aprender la extensión pgvector, pero reutiliza SQL ya conocido.	API propia más simple, pero es una herramienta nueva para el equipo.
Ajuste al problema	Búsqueda por similitud con índice HNSW, suficiente para el volumen de un corpus legal acotado.	Optimizado para búsqueda vectorial a gran escala, con más capacidad de la que este corpus requiere.

Decisión (Tabla 3): PostgreSQL + pgvector. La desventaja asumida es que el equipo debe administrar y optimizar el índice vectorial por su cuenta, sin la escalabilidad automática de un servicio gestionado, a cambio de no depender de un proveedor externo para un dato tan sensible como el corpus legal.

6.4 Decisión de renderizado: chat en el cliente, páginas estáticas

El núcleo del producto —la conversación con Articla— se implementa como una aplicación de página única: las respuestas deben aparecer sin recargar la página y el historial de la conversación debe mantenerse en pantalla. En concreto, dentro de Next.js el chat es una ruta renderizada en el cliente (comportamiento de SPA), mientras que el inicio, el aviso legal y la biblioteca de fuentes son páginas generadas de forma estática y por eso indexables en buscadores; ambos modos conviven en un mismo proyecto, por lo que elegir una SPA para el chat y usar Next.js no es contradictorio. La desventaja asumida es una carga inicial de JavaScript algo mayor que una aplicación multipágina pura, aceptable dado que el uso principal ocurre dentro de una sesión larga de conversación.

6.5 Plataforma móvil: web responsive y PWA en vez de app nativa

Tabla 4. Comparación de plataformas para el uso en móvil.

Criterio	Web responsive / PWA	App nativa (iOS y Android)
Curva de aprendizaje del equipo	Baja: se reutiliza el mismo código React de la versión web.	Alta: otro lenguaje o framework y proceso de publicación en tiendas.
Ajuste al problema	El producto es un chat de texto con citas: no necesita sensores ni funciones nativas, y una PWA se instala desde el navegador.	Daría acceso a funciones del dispositivo que el producto no necesita.
Costo de operación	Un solo código y un solo despliegue; sin cuentas de tiendas de aplicaciones.	Dos bases de código (o un framework adicional), más cuentas y revisión de las tiendas.

Decisión (Tabla 4): diseño responsive y PWA en lugar de app nativa, de modo que el curso (Móvil y Web) se cubra con una sola base de código. El prototipo ya adapta su menú a pantallas angostas mediante una media query. La desventaja asumida es el acceso limitado a funciones nativas avanzadas y la menor visibilidad que da una tienda de aplicaciones; se acepta porque el uso principal es escribir una pregunta y leer una respuesta.

6.6 Modelo de embeddings

Tabla 5. Comparación de modelos de embeddings.

Criterio	Modelo multilingüe de código abierto, en el servidor	Servicio de embeddings de un proveedor (API)
Curva de aprendizaje del equipo	Media: se usa con una librería de Python, pero hay que descargar y ejecutar el modelo.	Baja: una llamada HTTP.

Ajuste al problema	Soporta español; los modelos compactos entregan vectores de 384 dimensiones, la dimensión definida en el esquema de la base de datos.	Suele ofrecer mayor calidad en textos largos, pero los vectores quedan atados a ese proveedor.
Costo de operación	Sin costo por uso; consume CPU y memoria del servidor.	Costo por volumen de texto procesado y un segundo servicio externo del que depender.

Decisión (Tabla 5): modelo de embeddings multilingüe de código abierto, ejecutado en el servidor. La desventaja asumida es que su calidad con vocabulario jurídico en español puede ser menor que la de un servicio comercial y que obliga a hospedar el modelo; se compensa evaluando la recuperación con la batería de preguntas de prueba antes de fijar la dimensión definitiva (hoy 384 en db/schema.sql).

6.7 Modelo de lenguaje (generación de la respuesta)

Tabla 6. Comparación de modelos de lenguaje.

Criterio	Modelo comercial vía API	Modelo abierto autoalojado
Curva de aprendizaje del equipo	Baja: una llamada HTTP con el prompt y el contexto.	Alta: instalar, hospedar y ajustar un modelo, con hardware adecuado.
Ajuste al problema	Buena redacción en español y capacidad de seguir la instrucción de citar solo el contexto entregado.	Calidad variable en español jurídico; más difícil exigir citas fiables.
Costo de operación	Costo por uso, razonable para el volumen de un prototipo, y dependencia del proveedor.	Sin costo por uso, pero requiere un servidor con GPU o memoria elevada.

Decisión (Tabla 6): un modelo de lenguaje comercial consumido vía API, aislado tras una capa de servicio propia. El proveedor concreto se fijará en el Hito 2, comparando con la misma batería de preguntas de prueba la calidad de las respuestas, el cumplimiento de la instrucción de citar, el costo y la latencia (la promesa exige menos de un minuto). La desventaja asumida es el costo por uso y la dependencia de un tercero; se mitiga con la capa de servicio, que permite reemplazarlo, y enviándole solo la pregunta y los fragmentos recuperados, sin datos de identidad.

7. Prototipo navegable

El prototipo de este hito cubre el recorrido del caso de uso descrito en la sección 5.1 y las cinco secciones del mapa del producto: Inicio, Preguntar, Historial, Biblioteca de fuentes, Cuenta y Aviso legal, además del detalle de las fuentes citadas. Desde el inicio, la usuaria puede elegir una pregunta de ejemplo, ver la respuesta con sus citas, hacer clic en una cita para leer el texto literal del artículo, volver a la conversación, revisar el historial y la biblioteca, y borrar su historial desde Cuenta. El enlace del prototipo se encuentra en el archivo de instrucciones del repositorio (README.md). La Figura 3 muestra

el inicio; la Figura 4, el chat con sus citas; la Figura 5, el detalle de la fuente; la Figura 6, el historial; la Figura 7, la pantalla Cuenta; y la Figura 8, el aviso legal.

Limitaciones declaradas: es un prototipo de interfaz, sin motor de IA ni base de datos conectados. Solo la pregunta sobre el arriendo tiene una respuesta de demostración con citas; las otras dos preguntas de ejemplo y sus entradas del historial muestran un aviso de que su respuesta se incorporará con el motor del Hito 2; el botón Enviar acepta cualquier pregunta, pero responde con un aviso de demostración; y las fichas del Código del Trabajo, de la Ley 19.496 y de los libros indican que su carga está prevista para el Hito 2.

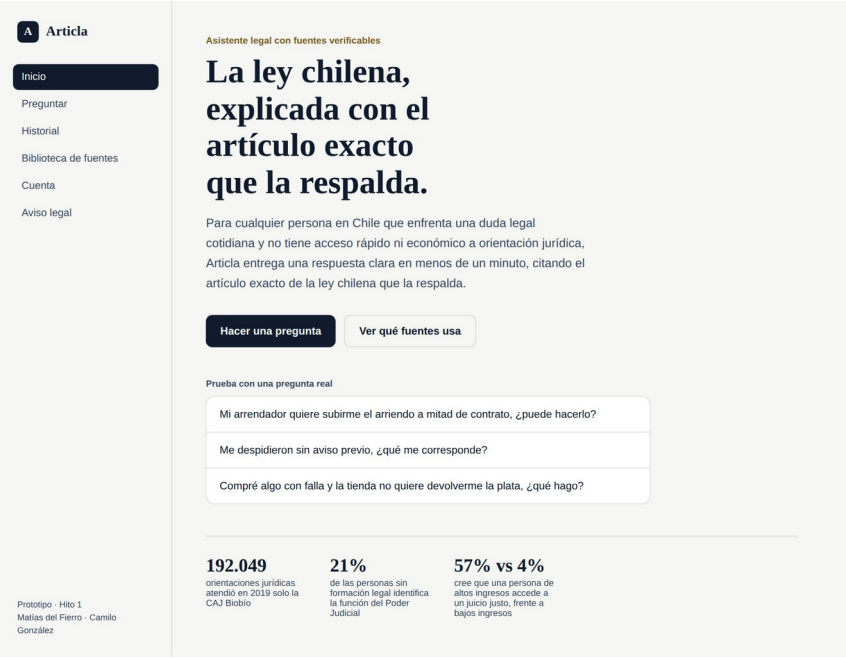


Figura 3. Pantalla de inicio: propuesta de valor, preguntas de ejemplo y cifras del problema.



Figura 4. Pantalla Preguntar: respuesta con sus citas clicables a los artículos de la ley.



Figura 5. Detalle de fuente: texto literal del artículo citado (Código Civil, art. 1545) y su origen en LeyChile.

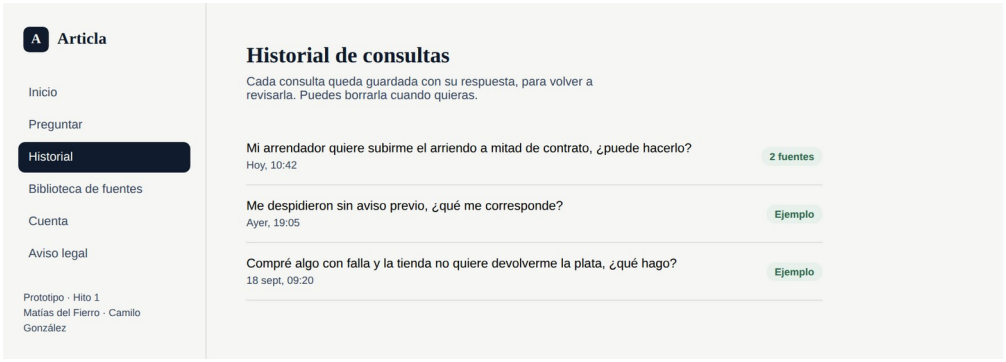


Figura 6. Historial de consultas: solo la primera tiene respuesta de demostración; las otras muestran un aviso.

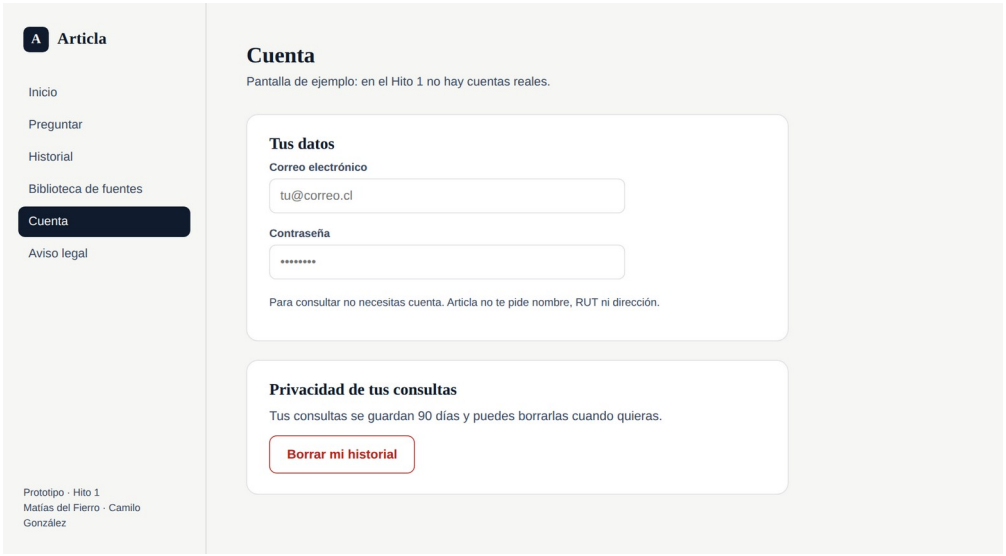


Figura 7. Pantalla Cuenta: datos de ejemplo y privacidad, con borrado del historial.



Figura 8. Aviso legal: Articla no reemplaza a un abogado y puede cometer errores.

8. Conclusión

Articla propone resolver un problema medible y documentado —la falta de orientación legal rápida y económica en Chile— con una arquitectura que separa claramente presentación, negocio y datos, y con una decisión de tecnologías fundamentada y no antojadiza. El prototipo navegable demuestra el flujo central del producto y la arquitectura de información sostiene ese flujo con secciones nombradas en el lenguaje del usuario. Los riesgos principales para el Hito 2 son técnicos —construir el motor de recuperación semántica y verificar la exactitud de las citas legales— y se abordarán con el mismo criterio de decisión documentada que se aplicó en este documento.

En este hito se logró fijar una promesa medible, dimensionar el problema con fuentes oficiales, diseñar la arquitectura y construir un prototipo navegable de seis pantallas. Queda pendiente todo lo que el prototipo solo simula: la API, la base de datos, la carga del corpus legal y la conexión con el modelo de lenguaje, que se construirán en los Hitos 2 y 3. Del proceso, el equipo aprendió que la exactitud de una cita legal debe comprobarse en la fuente y no suponerse (la primera cita del ejemplo de arriendo tuvo que corregirse, porque la ley especial no regula el reajuste de la renta); que una promesa con plazo y métrica obliga a diseñar la verificación desde el comienzo; y que la privacidad de las consultas es una decisión de diseño y no un detalle posterior.

Un riesgo adicional, no técnico, es de propiedad intelectual: los libros y apuntes de la Facultad de Derecho de la Universidad de Chile pueden estar protegidos por derechos de autor, a diferencia de los textos legales, que son de dominio público. Antes de construir el corpus definitivo en el Hito 2, el equipo gestionará con la Facultad el uso autorizado de ese material o priorizará fuentes abiertas equivalentes (Biblioteca del Congreso Nacional, repositorios académicos de acceso abierto).

Para que la promesa sea verificable en el Hito 3, el equipo fija desde ahora dos métricas de éxito: (1) tiempo de respuesta inferior a un minuto, medido de extremo a extremo, y (2) exactitud de las citas, es decir, el porcentaje de respuestas cuyo artículo citado existe y respalda realmente lo afirmado, comprobado contra una batería de preguntas de prueba revisadas manualmente con el texto oficial de la Biblioteca del Congreso Nacional, con una meta inicial de al menos 90%.

Finalmente, la redacción de las respuestas depende de un servicio externo (un modelo de lenguaje consumido vía API de un proveedor), lo que implica costo por uso, gestión de credenciales fuera del repositorio y exposición a cambios de precio o disponibilidad del proveedor. Para mitigarlo, la llamada al modelo se aislará tras una capa de servicio con interfaz propia, de modo que el modelo pueda reemplazarse sin modificar el resto del sistema.

Un riesgo adicional es la privacidad. El Historial guarda preguntas legales que pueden revelar información sensible sobre la vida de las personas (conflictos laborales, familiares o de vivienda). Por eso el diseño se limita a lo necesario: Articla no solicita nombre, RUT ni dirección para consultar; la tabla de consultas guarda solo el texto de la pregunta, la respuesta, una valoración opcional y la fecha; las consultas se conservan 90 días y la usuaria puede borrarlas antes (en la API, DELETE /consultas/{id}; en el prototipo, el botón «Borrar mi historial» de la pantalla Cuenta); el chat y el aviso legal piden no escribir datos personales; y al modelo de lenguaje externo solo se le envían la pregunta y los fragmentos recuperados, sin identificadores de la persona. En lo normativo, hoy rige la Ley 19.628; la Ley 21.719, publicada el 13 de diciembre de 2024, que crea la Agencia de Protección de Datos Personales (Ley 21.719, 2024), entra en vigencia el 1 de diciembre de 2026 según su texto, aunque a octubre de 2026 se tramita un proyecto que posterga esa fecha al 1 de diciembre de 2027 (Garrigues, 2026). El equipo diseña desde ya conforme a sus principios de finalidad, proporcionalidad y derecho de supresión, y confirmará la fecha de vigencia antes del Hito 3.

9. Bibliografía

Código Civil de Chile. (s. f.). Biblioteca del Congreso Nacional, LeyChile. https://www.bcn.cl/leychile

Corporación de Asistencia Judicial de la Región del Biobío. (2020). Informe resumen cuenta pública 2020. https://www.cajbiobio.cl/wp-content/uploads/2020/05/INFORME-RESUMEN-CUENTA-PUBLICA-2020.pdf

Garrigues. (2026, 11 de septiembre). Evolución de la regulación de datos personales en Chile: modernización del marco normativo, nueva institucionalidad y proyecciones. https://www.garrigues.com/es_ES/noticia/evolucion-regulacion-datos-personales-chile-modernizacion-marco-normativo-nueva

Ley 18.101. (1982, 29 de enero). Fija normas especiales sobre arrendamiento de predios urbanos. Biblioteca del Congreso Nacional, LeyChile. https://www.bcn.cl/leychile

Ley 21.719. (2024, 13 de diciembre). Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales. Diario Oficial de la República de Chile. https://www.bcn.cl/leychile

Ministerio de Justicia y Derechos Humanos y GfK Adimark Chile. (2015). Encuesta nacional de necesidades jurídicas y acceso a la justicia: Informe final. https://minjusticia.gob.cl/media/2025/04/Informe-final-Encuesta-Nacional-de-Necesidades-Juridicas-y-Acceso-a-Justicia.pdf

Open Government Partnership. (2021). Compromiso Chile: Acceso a la justicia (CL0075). https://opengovpartnership.org/members/chile/commitments/cl0075

Anexo A. Equipo y roles

La Tabla 7 resume quiénes integran el equipo y qué rol cumple cada uno.

Tabla 7. Integrantes del equipo, usuarios de GitHub y roles.

Integrante	Usuario de GitHub	Rol principal
Matías del Fierro	matias-nicolasdf21	Coordinación y repositorio
Camilo González	camilogonzalez114	Documento de propuesta y prototipo

Anexo B. Dominios de datos y fuentes previstas

- Código Civil y Código del Trabajo — texto oficial vigente desde LeyChile (Biblioteca del Congreso Nacional), de dominio público; es la fuente principal del corpus.
- Ley 18.101 (arrendamiento de predios urbanos) y Ley 19.496 (protección de los derechos de los consumidores) — Biblioteca del Congreso Nacional, LeyChile.
- Libros y apuntes de estudio de la Facultad de Derecho, Universidad de Chile — fuente complementaria, que se incorporará solo con autorización de uso o cuando esté disponible en acceso abierto.
- Fuentes adicionales a incorporar en el Hito 2, priorizando siempre texto legal vigente y verificable.

Anexo C. Declaración de uso de inteligencia artificial

Conforme a la regla de declaración de uso de inteligencia artificial del curso, el equipo detalla a continuación qué herramienta utilizó, en qué partes del trabajo y cómo verificó el resultado.

Herramienta utilizada: Claude (Anthropic), como asistente de redacción, diagramación y prototipado.

- Redacción y estructuración del documento de propuesta y de las láminas de presentación, a partir del problema, la promesa y las decisiones técnicas definidas por el equipo.
- Generación de los diagramas de arquitectura de información y cliente-servidor (Figuras 1 y 2), a partir de la estructura de pantallas y capas acordada por el equipo.
- Desarrollo del código HTML, CSS y JavaScript del prototipo navegable.

Cómo se verificó:

- Las cifras y referencias del capítulo 2 se comprobaron contra las fuentes originales citadas en la bibliografía.
- El texto de los artículos citados en el prototipo (Código Civil, art. 1545; Ley 18.101, art. 1) se cotejó con el texto de la ley publicado en línea.

- Ambos integrantes probaron manualmente cada pantalla del prototipo navegable para confirmar que el flujo de clics funciona como se describe en la sección 7, incluidas sus limitaciones declaradas.
- Además se ejecutó una prueba automatizada que recorre las seis pantallas y todos los botones y enlaces del prototipo, sin errores.
- Ambos integrantes revisaron el código y las decisiones de arquitectura para poder explicarlas ante cualquier pregunta durante la defensa.

No se cargó ni utilizó ningún dato personal real: los datos que aparecen en el prototipo (consultas de ejemplo, historial) son ficticios.